软件设计模式

项目考查报告

学　　院 软件工程学院

专　　业 软件工程

姓　　名 刘文彬

学　　号 2024081120　　班级软工 243 班

2026 年 11 月 1 日

目　录

1 考查题目.....1
2 项目概述.....2
2.1 背景与意义.....2
2.2 功能需求.....2
3 系统设计.....5
3.1 设计模式选择与应用分析.....5
3.2 数据库设计.....9
3.3 核心类设计.....14
4 系统开发.....17
4.1 技术选型与工程结构.....17
4.2 接口设计.....19
4.3 关键流程实现.....20
4.4 开发中遇到的几个问题.....22
5 测试与结果分析.....24
5.1 测试环境与方法.....24
5.2 功能测试.....24
5.3 边界与异常测试.....26
5.4 并发与性能测试.....27
5.5 结果分析.....28
6 项目开源.....29
7 总结与展望.....30
参考文献.....32

1 考查题目

题目一：校园一卡通消费与结算系统。

校园一卡通是高校中使用频率最高的信息系统之一：食堂、超市、开水房、澡堂、图书馆门禁、宿舍水控都依赖它完成身份识别与费用结算。系统需要覆盖以下五个方面的功能：

（1）账户与卡片管理：支持学生开户、卡片挂失与解挂、销户、充值以及账户状态查询；账户与卡片之间的状态变更要保持一致，同一个事务内完成。

（2）消费扣款：支持食堂 POS、超市收银、门禁、水控等多种终端；不同终端上报的报文格式并不相同；消费时可以叠加普通、折扣、补贴、积分抵扣等多种优惠。

（3）交易与结算：记录充值、消费、退款三类交易；需要处理并发扣款、重复消费（幂等）以及退费冲正，并支持按终端、按日期进行日终结算。

（4）查询与统计：交易明细查询、按终端与时间维度的对账、流水统计分析。

（5）消息与通知：交易完成后向对账、统计、消息推送等关注方发送通知。

题目要求采用分层架构（表示层 / 业务层 / 数据访问层或 MVC），至少使用三种设计模式，提供 Web 前端界面，并完成功能测试、边界测试、异常测试以及并发或性能测试，保留测试记录。

2 项目概述

2.1 背景与意义

校园一卡通系统看起来只是一个“扣钱”的程序，真正做起来却处处是坑：食堂窗口的 POS 上报的是消费金额，宿舍水控上报的是用水升数，图书馆门禁干脆没有金额，只表示“放行一次”；同一个账户可能在一秒内被两台终端同时扣款；一笔消费失败以后要能撤销，而撤销不能把原来的流水改掉，否则对账就永远对不平；优惠规则每学期都在变，今天打八五折，明天加两元补贴，后天又允许积分抵扣。这些需求有一个共同点——业务规则在不停地变，而系统的骨架必须稳住。

这正是设计模式要解决的问题。选择这个题目，一方面是因为它的业务场景贴近真实，需求文档里写到的每一条都能在校园里找到对应；另一方面是因为它的变化点足够清晰：优惠算法会变、终端种类会变、校验规则会变、通知方式会变。把这些变化点识别出来，用接口把它们和稳定的交易主流程隔离开，设计模式的引入就有了真实的理由，而不是为了凑数在代码里硬塞几个 if-else 包装成策略模式。

本项目的目标是在一个可运行的 Web 系统里，把策略、装饰器、责任链、工厂、建造者、观察者、适配器、代理、单例九种模式落到具体的业务位置上，每一个模式都能回答三个问题：它替换掉了哪一段本会写死的代码、它让什么样的修改不再需要改动主流程、它在运行时真正被调用了几次。

2.2 功能需求

系统面向三类使用者：持卡学生、部署在商户与宿舍的终端设备、负责开户与对账的管理员。用例关系如图 2-1。

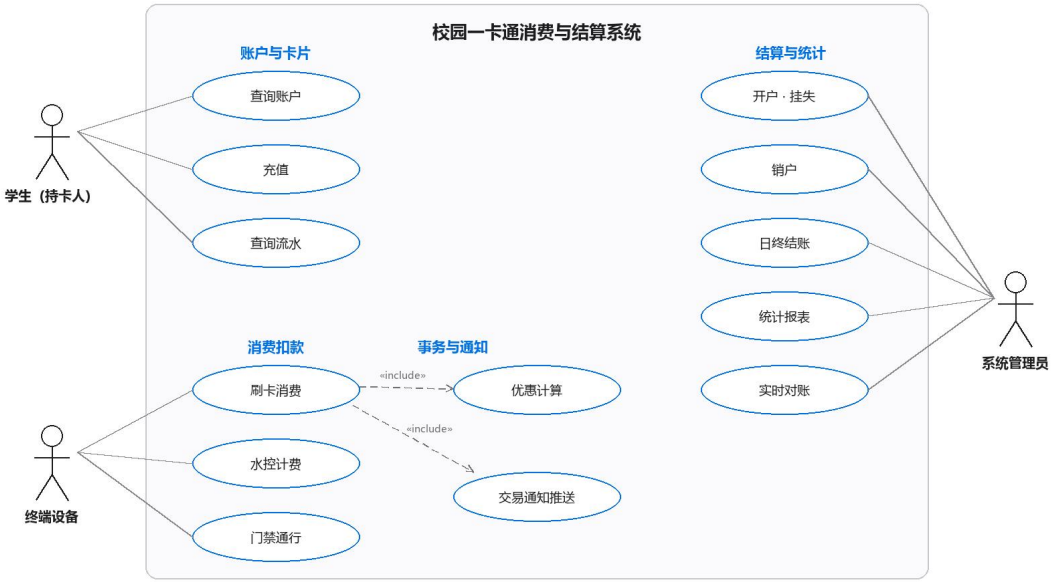


图 2-1 系统用例图

按用例整理出的功能需求如表 2-1。

表 2-1 功能需求

模块	功能	需求说明
账户与卡片	开户	学号唯一；开户同时生成卡号，默认支付密码 123456
	挂失 / 解挂	账户状态与卡片状态在同一事务内同步变更
	销户	卡内余额必须为 0，销户后不可恢复
	充值	单笔 1～5000 元，幂等
	修改支付密码	原密码校验，新密码必须为 6 位数字
消费扣款	POS 消费	上报金额；食堂、超市共用一套报文字段
	水控计费	上报用水升数，按终端单价换算金额
	门禁通行	上报空报文，按次计费
	优惠计价	普通 / 折扣 / 补贴 / 积分四类，可叠加，优惠封顶
交易与结算	退款冲正	新增反向流水，原单标记已冲正，原金额不改

	幂等控制	requestNo 全局唯一，重复提交返回原交易号
	日终结账	按终端汇总当日消费笔数、金额与退款
查询与统计	流水查询	支持卡号、类型、终端、时间区间与分页
	统计报表	七日趋势、类型分布、终端汇总
	实时对账	交易完成即累计，不查库即可看到当日实时口径
消息通知	交易通知	消费、充值、退款完成后推送给对账、统计、通知、积分四个关注方

除功能需求外，还有四条必须满足的约束，它们直接决定了后面的设计：

- 并发安全：同一账户的余额在任意并发下都不能被扣成负数。
- 事务一致：账户、卡片、流水三者要么全部成功，要么全部回滚。
- 可扩展：新增一种优惠规则或一种终端类型时，不改动交易主流程的代码。
- 可追溯：每一次操作都有耗时与结果记录，流水只追加不修改。

3 系统设计

3.1 设计模式选择与应用分析

3.1.1 稳定点与变化点

在动手写模式之前，先把系统里“不会变的东西”和“一定会变的东西”分开列出来。表 3-1 是一次完整梳理的结果，后面的每一个模式都必须挂在这张表的某一行上，否则就不引入。

表 3-1 稳定点与变化点分析

变化点	变化频率	典型变化	应对模式	被固定下来的接口
优惠算法	高（每学期）	打折、满减、补贴、积分抵扣、封顶	策略 + 装饰器	PricingStrategy
终端报文	中（逐年新增）	POS 报金额、水控报升数、门禁报空报文	适配器	TerminalAdapter
支付校验规则	中	挂失校验、密码校验、单笔限额、日限额、余额校验	责任链	PayHandler
交易类型	低	消费、充值、退款，将来可能加转账	工厂	AbstractTransaction
交易单据内容	低	单据要带明细、步骤、优惠说明	建造者	TransactionTicketBuilder
通知接收方	高	对账、统计、消息推送、积分奖励	观察者	TradeObserver
横切逻辑	中	限流、耗时统计、操作日志	代理	PayFacade
全局参数	低	限额、积分兑换比例、流水号	单例	SystemConfig

稳定点是那条贯穿始终的主线：校验 → 计价 → 扣款 → 落库 → 通知。不管优惠怎么变、终端怎么增，这五个环节的顺序不会变，账户与流水的数据结构不会变。系统的设计就是围绕这条主线展开的：主线写在 CardServiceImpl 里，只依赖接口；

所有会变的部分被推到接口的后面，由模式负责装配。这正是“开闭原则”在具体业务上的一次落地。

图 3-1 把这九种模式放回了业务流上：一笔刷卡消费从进入系统到写出流水，依次经过代理、适配器、策略与装饰器、责任链，最后由工厂、建造者与观察者收尾。



图 3-1 一笔交易经过的设计模式

3.1.2 九个模式的落点

下面对九个模式逐一说明：它解决了什么问题、在系统里扮演什么角色、对应哪个类、以及拿掉它会退化成什么样子。

（1）策略模式 —— 优惠算法。优惠是最典型的多算法场景。普通按原价、折扣按比例、补贴减固定金额、积分按比例抵扣，四种算法输入相同（金额、积分、规则参数）、输出相同（应付金额、优惠额、积分变化），但计算过程完全不同。如果写成枚举加 switch，每加一种规则就要改一次 switch，而且方法会越来越长。这里定义 PricingStrategy 接口，四个实现类各自封装一种算法，PricingContext 用一个 EnumMap 按规则类型取策略，调用方只看到 calculate(PricingRequest) 一个方法。新增规则只需要新增一个类并加上 @Component，交易主流程一行都不用改。

（2）装饰器模式 —— 优惠叠加。规则往往不是单选：一张卡既享受八五折，又能用积分抵扣，还要保证优惠总额不超过 20 元。如果靠策略模式在内部组合，PricingStrategy 会因为承载“组合逻辑”而变复杂，而且组合顺序无法在运行时调整。这里让 PricingDecorator 实现 PricingStrategy 并持有一个 inner 策略，

CapDecorator、SubsidyDecorator、PointsDecorator 分别在自己的 calculate 里先调用 inner，再对结果做一次加工。PricingService 拿到基准策略后，按规则串的顺序一层层包上去，最后恒定包一层封顶装饰器。装饰器和被装饰者类型一致，所以调用方完全感知不到自己被包了几层。实测 100 元在“八五折 + 补贴 2 元”下算出 83.00 元，正是 100×0.85-2 的结果。

（3）责任链模式 —— 支付校验。一笔消费在真正扣钱之前要过六道关：卡片是否挂失、密码是否正确、是否超过单笔限额、是否超过日限额、余额是否充足、扣款是否成功。写成六个连续的 if，任何一道要调整顺序或临时下线都很别扭；而且这六件事的关注点完全不同，混在一个方法里会让扣款逻辑难以阅读。这里把每道校验做成一个 PayHandler，每个 handler 持有 next 指针，抽象类里的 doHandle 先执行自己的 check，再交给下一个。PayChainFactory 在构造时把六个 handler 按顺序串成一条链。链的好处是顺序在装配处一目了然，增删某一环只要改装配代码，且每个 handler 可以单独测试。最后一环 DeductHandler 直接执行带条件的更新语句，扣减不成功就抛出业务异常，整条链随之中断。

（4）工厂模式 —— 交易类型。消费、充值、退款三类交易对账户的影响方向不同：消费是扣减、充值是增加、退款也是增加，但退款还要校验原单。把“这类交易该怎么改账户”这件事交给 AbstractTransaction 的三个子类，TransactionFactory 用 EnumMap 注册后按类型返回对应对象。这样 CardServiceImpl 在编排时不需要知道具体是哪一类交易，只调用 apply(mapper, accountId, amount)。将来若增加“转账”，只需新增一个子类并注册，编排代码不变。

（5）建造者模式 —— 交易单据。一条交易流水有十几个字段，其中交易号、请求号、卡号、账户号、终端、原价、实付、优惠、策略编码、状态、原交易号、备注需要按业务不同分别赋值。用构造函数会变成一长串参数，用 setter 又无法保证“字段齐全才落库”。这里用 TransactionTicketBuilder 提供链式方法，最后由 build() 一次性产出单据；TransactionDirector 再在其上定义三种标准装配流程

（消费单、充值单、退款单），每一步自动往单据里追加一条处理说明。这些步骤文本后来直接在界面的交易详情里展示，相当于免费得到了一份可视化的调用链。

（6）观察者模式 —— 交易通知。交易完成之后有一堆“顺便要做”的事：把金额累计到当日对账口径、给统计模块加一笔、生成一条推送消息、按消费金额送积分。这四件事彼此独立，任何一件失败都不应该影响交易本身。如果都写在 CardServiceImpl 里，方法会变成流水账，而且每次新增一个关注方都要改主流程。这里定义 TradeObserver 接口和 TradeEventPublisher 主题，四个观察者各自实现 onTrade，发布者遍历列表逐个回调，单个观察者抛异常只记录警告并继续。这是典型的发布—订阅，把“交易发生了”和“谁来关心”彻底解耦。界面上那张实时对账卡片的数据，就是观察者内存计数的结果，不查数据库也能看到。

（7）适配器模式 —— 终端报文。三种终端的报文结构完全不同：POS 直接给金额，水控给的是用水升数、金额要乘以终端单价，门禁只发一个空报文、按次收费。若不统一，交易主流程里就得写三个分支。这里定义 TerminalAdapter 接口，三个实现分别把各自的原始报文翻译成统一的 PayRequest（含金额与一行换算说明），TerminalAdapterRegistry 按终端类型分派。主流程只面对 PayRequest，新增一种终端（比如洗衣房刷卡机）只需新增一个适配器并注册。水控那一行“20 升 × 0.50 元/升 = 10.00 元”的换算说明，最后原样出现在流水备注里，便于事后核对。

（8）代理模式 —— 限流与操作日志。限流、耗时统计、操作日志这三件事和业务本身无关，却必须包在每一次交易外面。直接写进 CardServiceImpl 会污染业务代码，用 Spring AOP 又难以在答辩时讲清楚调用关系。这里用最朴素的方式：定义 PayFacade 接口，CardServiceImpl 实现它，CardServiceProxy 也实现它并持有 CardServiceImpl 的引用，标记为 @Primary 后注入到 Controller 的就是代理对象。代理在调用真实对象前后做限流判断、计时和日志落库，业务代码对此毫无感知。这也解释了压测里为什么 25 笔并发只有 20 笔成功——每秒每卡 20 笔的限制正是这一层拦下的。

（9）单例模式 —— 全局参数与流水号。单笔限额、日限额、积分兑换比例这

类全局参数，以及交易流水号生成器，都要求进程内只有一份实例。Spring 的 Bean 默认就是单例，但这里仍然用静态内部类 Holder 手写了一遍 SystemConfig 和 TradeNoGenerator，目的是把“为什么必须是单例”这件事显式表达出来：配置被多处读取，多份实例会导致阈值不一致；流水号若有多份生成器，序号会重复，交易号就撞了。流水号采用“LWB + 时间戳 + 四位自增序号”的格式，序号用 AtomicInteger 保证线程安全。

3.1.3 选型中的取舍

有几个地方本来可以用更省事的写法，最后没有采用，理由记录如下。

- 优惠叠加没有用“策略组合”而是用装饰器：策略的语义是“选一个算法”，装饰器的语义是“在算法外面加一层”，两者职责不同。叠加属于后者，用装饰器能让每一层只关心自己那一次加工，顺序由装配处决定。
- 校验没有用规则引擎或注解校验：六道校验里有三道需要查库（日限额、余额、扣款），注解校验只能处理入参格式，处理不了这种带状态的判断。责任链能同时容纳纯内存判断和数据库操作。
- 终端适配没有在 Controller 里做字段转换：转换逻辑一旦散落在接口层，新增终端就要改接口层，而下单流程本身并没有变化。适配器把变化收敛到一个类里。
- 代理没有用 Spring AOP：AOP 的切点表达式对初学者来说不直观，答辩时也不容易讲清楚“谁调用了谁”。手写代理虽然代码多了一点，但调用关系在类图上一目了然。

3.2 数据库设计

数据库共 8 张表，围绕“账户—卡—交易—终端”这条主线组织。账户与卡是一对一，但拆成两张表：账户持有余额与积分，是资金的主体；卡持有卡号与密码，是介质。分开的好处是换卡、补卡不影响账户，挂失也可以精确到卡。实体关系如图 3-2。

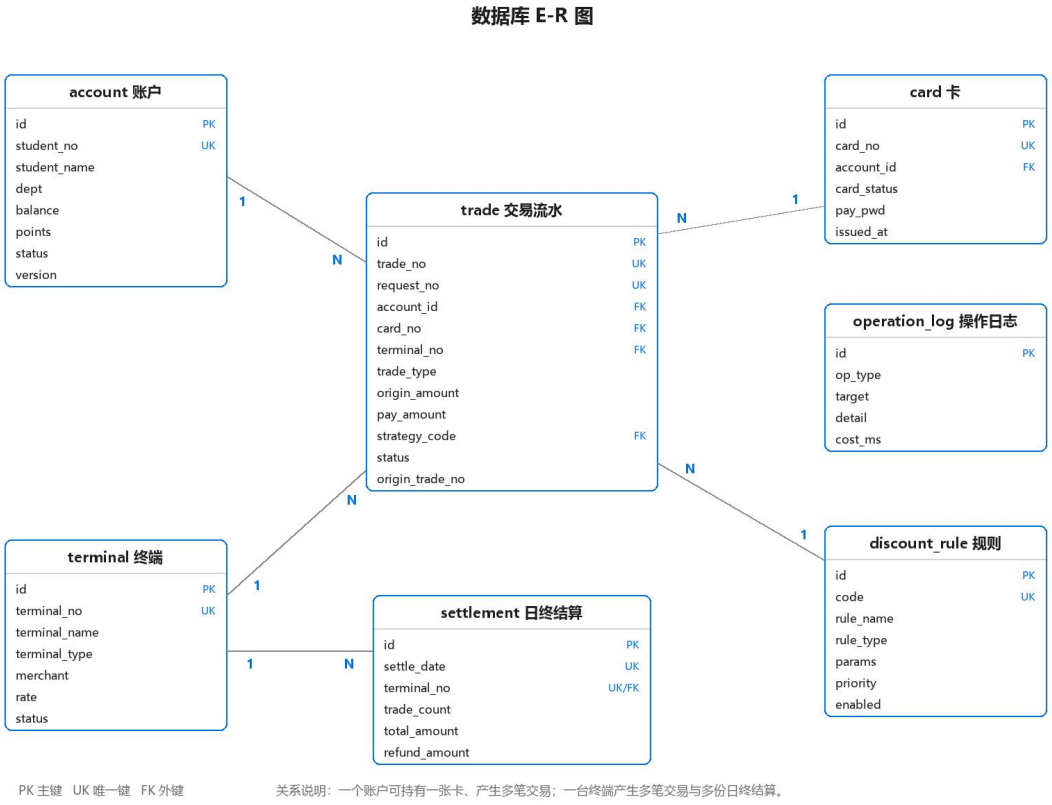


图 3-2 数据库 E-R 图

各表结构如下。金额统一使用 DECIMAL(12,2)，不使用浮点数，避免 0.1+0.2 这类精度问题在账目上累积。交易表中同时保留了原价、优惠额、实付额三个字段，对账时可以直接用 SQL 汇总，不必再回算。

表 3-2 account 账户表

字段	类型	注释	外键关系	备注
id	BIGINT	账户主键	—	自增
student_no	VARCHAR(20)	学号	—	唯一索引
student_name	VARCHAR(32)	姓名	—	
dept	VARCHAR(64)	院系	—	
balance	DECIMAL(12,2)	账户余额	—	默认 0.00，扣减时校验不为负
points	INT	积分	—	默认 0，消费一元记一分

status	TINYINT	账户状态	—	1 正常 / 2 挂失 / 3 销户
version	INT	版本号	—	预留给乐观锁
created_at	DATETIME	创建时间	—	默认当前时间

表 3-3 card 卡表

字段	类型	注释	外键关系	备注
id	BIGINT	卡主键	—	自增
card_no	VARCHAR(20)	卡号	—	唯一索引，开户时按账户号生成
account_id	BIGINT	所属账户	→ account.id	一张卡对应一个账户
card_status	TINYINT	卡状态	—	1 正常 / 2 挂失 / 3 注销
pay_pwd	CHAR(6)	支付密码	—	默认 123456
issued_at	DATETIME	发卡时间	—	

表 3-4 terminal 终端表

字段	类型	注释	外键关系	备注
id	BIGINT	终端主键	—	自增
terminal_no	VARCHAR(32)	终端编号	—	唯一索引，如 POS-001
terminal_name	VARCHAR(64)	终端名称	—	如第一食堂 3 号窗口
terminal_type	VARCHAR(16)	终端类型	—	POS / WATER / GATE
merchant	VARCHAR(64)	所属商户	—	
rate	DECIMAL(6,2)	单价	—	水控为每升单价，门禁为每次费用
status	TINYINT	状态	—	1 在线 / 0 停用

表 3-5 trade 交易流水表

字段	类型	注释	外键关系	备注
id	BIGINT	流水主键	—	自增
trade_no	VARCHAR(32)	交易号	—	唯一索引，LWB + 时间戳 + 序号
request_no	VARCHAR(64)	请求号	—	唯一索引，幂等控制键
account_id	BIGINT	账户	→ account.id	
card_no	VARCHAR(20)	卡号	→ card.card_no	冗余存储，便于按卡查询
terminal_no	VARCHAR(32)	终端编号	→ terminal.terminal_no	充值为空
trade_type	VARCHAR(16)	交易类型	—	RECHARGE / CONSUME / REFUND
origin_amount	DECIMAL(12,2)	原价	—	
discount_amount	DECIMAL(12,2)	优惠金额	—	
pay_amount	DECIMAL(12,2)	实付金额	—	
points_delta	INT	积分变化	—	负数为抵扣
strategy_code	VARCHAR(64)	优惠规则	→ discount_rule.code	可为多条，逗号分隔
status	TINYINT	状态	—	1 成功 / 2 失败 / 3 已冲正
origin_trade_no	VARCHAR(32)	原交易号	→ trade.trade_no	仅退款单有值
remark	VARCHAR(255)	备注	—	记录适配与换算说明

表 3-6 discount_rule 优惠规则表

字段	类型	注释	外键关系	备注
id	BIGINT	规则主键	—	自增
code	VARCHAR(32)	规则编码	—	唯一索引，如 DISC-85
rule_name	VARCHAR(64)	规则名称	—	

rule_type	VARCHAR(16)	规则类型	—	NORMAL / DISCOUNT / SUBSIDY / POINTS
params	VARCHAR(64)	规则参数	—	折扣率、补贴额或兑换比例
priority	INT	优先级	—	叠加顺序
enabled	TINYINT	是否启用	—	1 启用 / 0 停用

表 3-7 settlement 日终结算表

字段	类型	注释	外键关系	备注
id	BIGINT	结算主键	—	自增
settle_date	DATE	结算日期	—	与终端编号组成唯一键
terminal_no	VARCHAR(32)	终端编号	→ terminal.terminal_no	
trade_count	INT	消费笔数	—	
total_amount	DECIMAL(12,2)	消费总额	—	
refund_amount	DECIMAL(12,2)	退款总额	—	
status	TINYINT	状态	—	1 已结算

表 3-8 operation_log 操作日志表

字段	类型	注释	外键关系	备注
id	BIGINT	日志主键	—	自增
op_type	VARCHAR(32)	操作类型	—	CONSUME / RECHARGE / REFUND
target	VARCHAR(64)	操作对象	—	卡号
detail	VARCHAR(255)	详情	—	成功或失败原因
cost_ms	INT	耗时	—	代理层计时写入

created_at	DATETIME	发生时间	—	

两处设计在后续开发中被证明是必要的：一是 trade 表的 request_no 唯一索引，前端网络抖动导致的重复提交会直接撞在索引上，被业务层翻译成“该请求已处理”而不是产生两笔扣款；二是 account 表保留了 version 字段，虽然当前采用的是带条件的原子更新，但这个字段为后续切换乐观锁留下了余地。

3.3 核心类设计

系统按表示层、接口层、业务层、设计模式层、数据访问层划分，图 3-3 是围绕交易编排展开的核心类图，每个类上方标注了它在模式中扮演的角色。

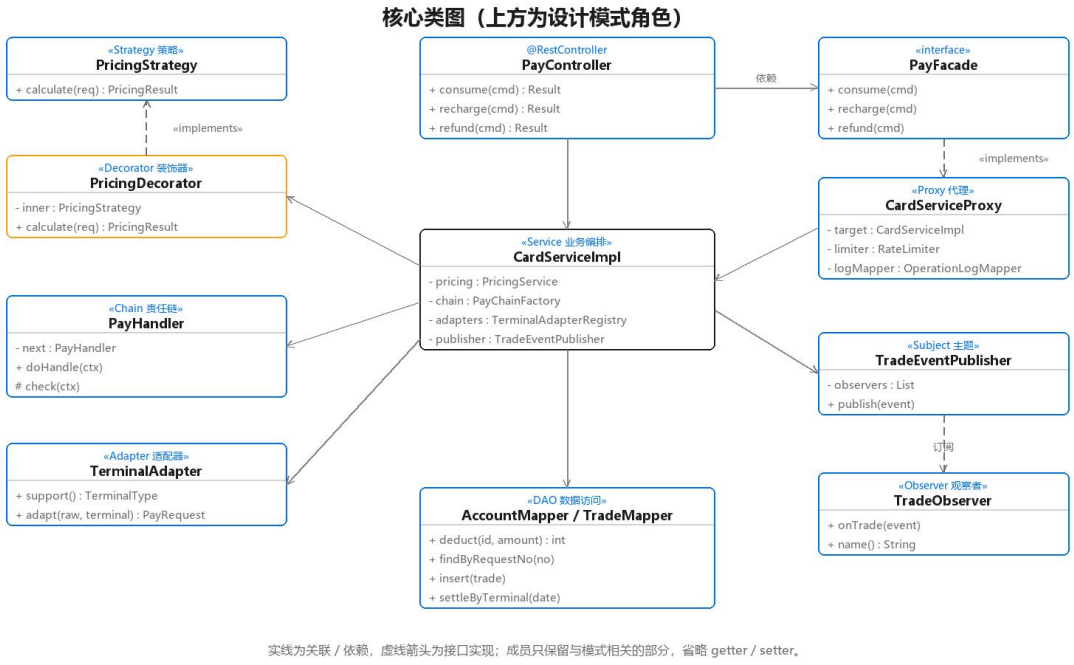


图 3-3 核心类图

整个交易主流程由 CardServiceImpl 编排，它只依赖接口，不依赖任何具体实现：计价依赖 PricingStrategy，校验依赖 PayHandler 链，报文转换依赖 TerminalAdapter，通知依赖 TradeObserver，而这些具体对象都由 Spring 在启动时注入。表 3-9 列出了主要类的职责划分。

表 3-9 核心类职责

类 / 接口	所在层	模式角色	职责
PayController	接口层	—	接收 HTTP 请求，补齐 requestNo，统一封装返回
PayFacade	业务层	被代理接口	交易能力的抽象，供接口层调用
CardServiceProxy	业务层	代理	限流、计时、写操作日志，再委派真实业务
CardServiceImpl	业务层	编排者	组装适配、计价、校验、落库、通知五个环节
PricingService	业务层	—	按规则编码串起基准策略与各层装饰器
PricingStrategy	模式层	策略	统一的计价入口
PricingDecorator	模式层	装饰器	持有 inner 策略，对结果做二次加工
PayHandler	模式层	责任链节点	单项校验，通过后交给下一环
PayChainFactory	模式层	责任链装配	按顺序串起六道校验
TerminalAdapter	模式层	适配器	把终端原始报文翻译成统一请求
AbstractTransaction	模式层	工厂产品	一类交易对账户的操作
TransactionTicketBuilder	模式层	建造者	链式组装交易单据
TransactionDirector	模式层	导演者	预置消费、充值、退款三种装配流程
TradeEventPublisher	模式层	主题	维护观察者列表并广播交易事件
TradeObserver	模式层	观察者	对账、统计、通知、积分四个实现
SystemConfig / TradeNoGenerator	模式层	单例	全局参数与流水号
AccountMapper 等	数据访问层	—	注解式 SQL，含带条件的原子扣减

值得单独说明的是扣款语句。余额校验在责任链的 BalanceHandler 里做了一次，但那次校验只是“读”；真正防止超扣的是 DeductHandler 里这一条带条件的更新：

```
UPDATE account SET balance = balance - #{amount}, version = version + 1
```

```
WHERE id = #{id} AND balance >= #{amount}
```

数据库对同一行的更新是串行的，两个并发请求只有一个能拿到行锁并满足 balance >= amount，另一个的影响行数为 0，业务层据此抛出“余额不足（并发扣减失败，请重试）”。这条语句把“判断”和“扣减”合并成一个原子操作，是并发安全的关键，也是后面并发测试能够通过的原因。

4 系统开发

本章按“选型—结构—接口—关键流程—问题排查”的顺序说明后端的开发过程。前端只承担演示职责，因此这里只在必要处提及，重点放在后端的接口设计与实现。

4.1 技术选型与工程结构

技术选型受两个条件约束：一是课程要求使用 Web 前端与关系型数据库，二是本机开发环境处于离线状态，无法联网下载依赖。因此所有组件都选用本地 Maven 仓库中已有的版本，前端的构建工具也换成了免构建的方案。

表 4-1 技术选型

层次	技术	版本	选择理由
运行环境	JDK	17	Spring Boot 3.x 的最低要求
后端框架	Spring Boot	3.2.7	本地仓库中唯一可离线使用的稳定版本
持久层	MyBatis	3.5.16	注解式 SQL，事务与映射可控，比 JPA 更贴近手写 SQL 的教学习惯
数据库	MySQL	8.0	课程统一要求；支持行锁与条件更新，适合实现原子扣减
连接池	HikariCP	5.0.1	Spring Boot 默认集成
前端框架	Vue	3.5	组合式 API，思路清晰，代码量小
前端构建	原生 ES Module	—	离线环境无法安装打包工具，改用浏览器直接加载运行时
文档生成	python-docx	1.2	按模板生成报告，图表由 Pillow 绘制

系统的分层结构如图 4-1。表示层是浏览器里的单页应用，通过 HTTP 调用接口层；接口层只做参数接收与结果封装，不写业务判断；业务层负责编排；设计模式层

是把九种模式集中放置的一层，它被业务层依赖，自身只依赖数据访问层的接口。这样分层的好处是模式代码与业务代码物理隔离，修改某一层不会牵连其他层。

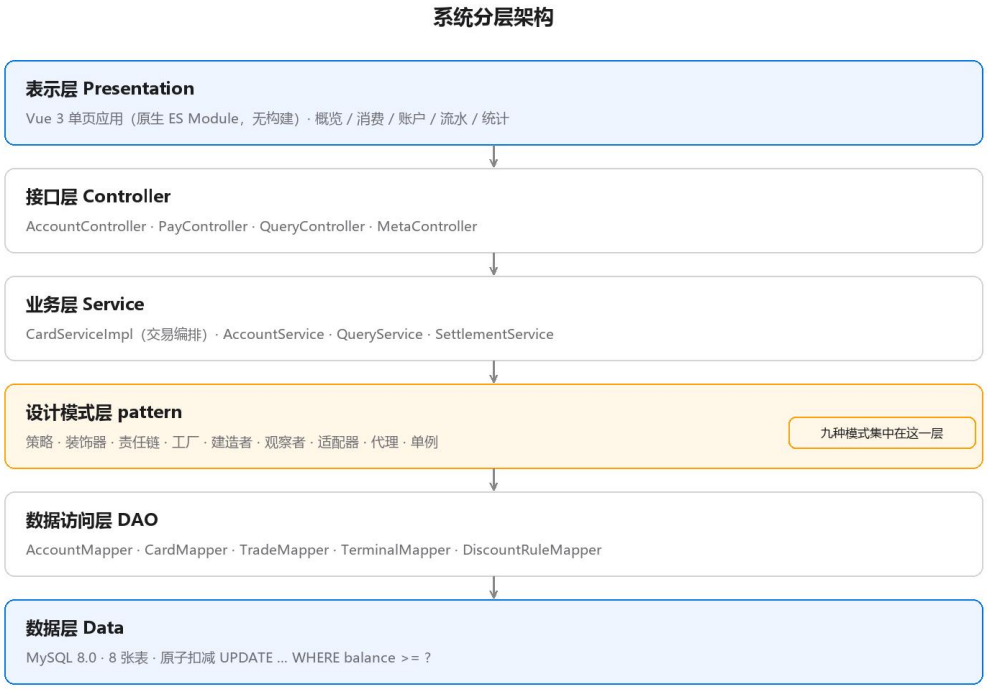


图 4-1 系统分层架构

工程采用标准的分层目录，包名以 com.lwb.campuscard 为根，各层的职责与上图一致：

```
backend/src/main/java/com/lwb/campuscard/
├── common/          统一返回体、业务异常、错误码
├── config/          MyBatis、跨域、时间序列化配置
├── controller/      PayController、AccountController、QueryController、MetaController
├── service/         CardServiceImpl、AccountService、QueryService、SettlementService
├── pattern/
│   ├── strategy/    PricingStrategy 与四个计价实现
│   ├── decorator/   PricingDecorator 与三个装饰器
│   ├── chain/       PayHandler 与六道校验
│   ├── factory/     AbstractTransaction 与工厂
│   ├── builder/     交易单据建造者与导演者
│   ├── observer/    事件发布者与四个观察者
```

```
│   ├── adapter/      终端适配器与注册表
│   ├── proxy/        业务代理与限流器
│   └── singleton/    全局配置与流水号生成器
├── dao/              七个 Mapper 接口
└── model/            实体、枚举、DTO
```

设计模式单独成包，是为了让“模式”这件事在工程结构上就能被看见。评审时打开 pattern 目录，九个包对应九个模式，每个包内部是接口加若干实现，不掺杂业务编排代码。

4.2 接口设计

接口统一使用 REST 风格，路径按资源划分，返回体统一为 { code, message, data } 三段式：code 为 0 表示成功，非 0 为业务错误码，message 是可以直接展示给用户的中文提示，data 为业务数据。前端因此只需要判断 code，不必关心 HTTP 状态码的差异，异常提示也能原样弹出。全部接口见表 4-2。

表 4-2 后端接口清单

方法	路径	说明	主要参数
POST	/api/accounts	开户	学号、姓名、院系、密码
GET	/api/accounts	账户与卡片列表	—
GET	/api/accounts/{cardNo}	账户详情	卡号
POST	/api/accounts/{cardNo}/status	挂失 / 解挂 / 销户	目标状态
POST	/api/accounts/{cardNo}/password	修改支付密码	原密码、新密码
POST	/api/pay/consume	消费扣款	终端号、卡号、报文、优惠规则、密码
POST	/api/pay/recharge	充值	卡号、金额、请求号
POST	/api/pay/refund	退款冲正	原交易号、请求号
GET	/api/trades	流水分页查询	卡号、类型、终端、起止时间、页码

GET	/api/stats/overview	总览指标	—
GET	/api/stats/trend	消费趋势	天数
GET	/api/stats/type	交易类型分布	—
GET	/api/stats/terminal	终端消费汇总	—
GET	/api/notifications	交易通知列表	—
GET	/api/reconciliation	实时对账快照	—
GET	/api/logs	操作日志	条数
GET	/api/terminals	终端列表	—
GET	/api/rules	优惠规则列表	—
GET	/api/config	系统参数	—
POST	/api/settlement/run	执行日终结账	日期
GET	/api/settlement	查询结算结果	日期

有几个细节是在联调中逐步定下来的。其一，requestNo 由前端生成，后端在为空时补一个 UUID，这样既保证幂等键始终存在，也不强制前端一定要传。其二，消费接口的报文放在 payload 里由适配器解析，而充值与退款直接用顶层字段，因为后两者不涉及终端报文差异。其三，分页接口的 size 上限锁定为 100，避免前端误传大数值把整表查出来。

4.3 关键流程实现

一笔刷卡消费要穿过代理、适配器、计价、责任链、工厂与观察者五个环节，调用关系如图 4-2。图中实线是调用，虚线是返回，箭头上的文字是实际传递的方法名。

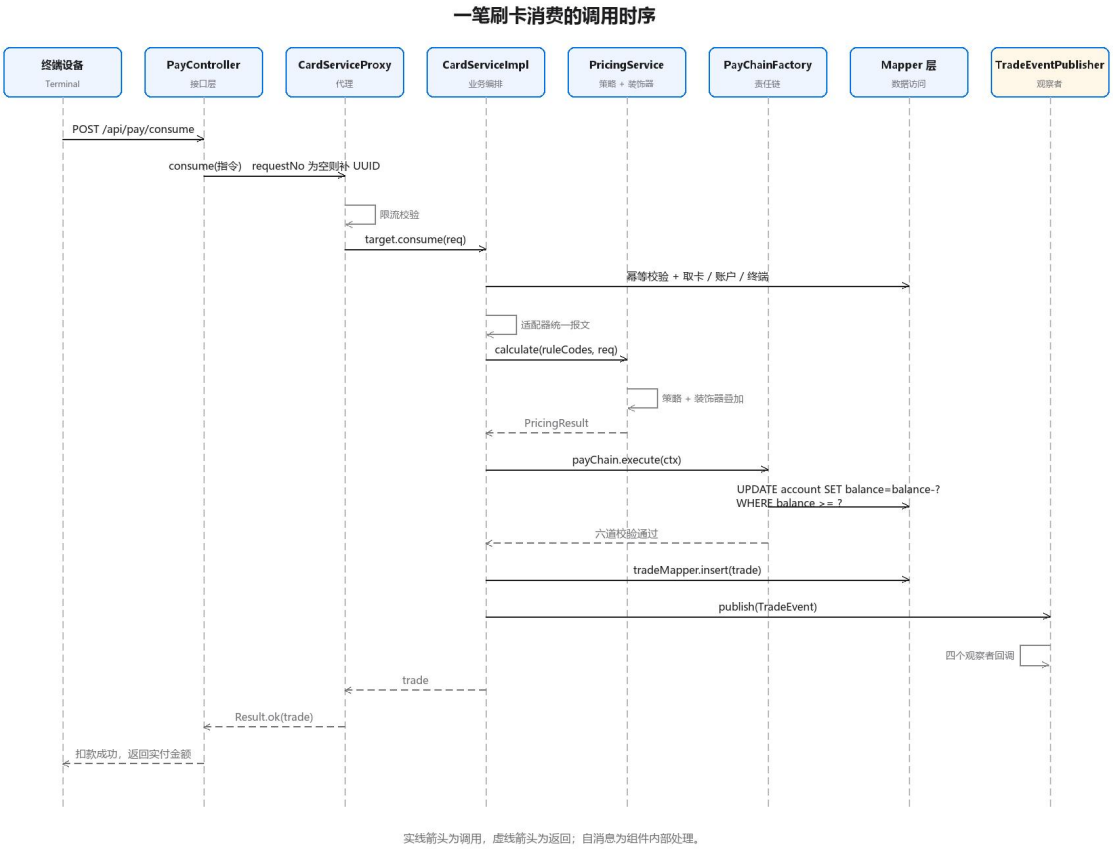


图 4-2 消费交易的调用时序

第一个环节是代理。CardServiceProxy 标注为主实现，控制器注入的是 PayFacade 接口，拿到的却是代理对象，业务本身并不知道自己被加了限流与日志：

```
public Trade consume(ConsumeCommand cmd) {
    if (!limiter.tryAcquire(cmd.getCardNo())) {
        throw new BizException("操作太频繁，请稍后再试");
    }
    long start = System.currentTimeMillis();
    try {
        Trade trade = target.consume(cmd);
        logMapper.insert("CONSUME", cmd.getCardNo(), "成功 " + trade.getTradeNo(),
                (int) (System.currentTimeMillis() - start));
        return trade;
    } catch (RuntimeException e) {
        logMapper.insert("CONSUME_FAIL", cmd.getCardNo(), e.getMessage(),
                (int) (System.currentTimeMillis() - start));
```

```
            throw e;
        }
    }
```

第四个环节是责任链。六个校验节点各自只关心一件事，通过后交给下一环，任何一环抛出业务异常，后面的节点都不会执行。链的抽象只有三个方法：

```
public abstract class PayHandler {
    private PayHandler next;
    public PayHandler next(PayHandler next) {
        this.next = next;
        return next;
    }
    public void doHandle(PayContext ctx) {
        check(ctx);
        if (next != null) {
            next.doHandle(ctx);
        }
    }
    protected abstract void check(PayContext ctx);
}
```

第三个环节是计价。装饰器让优惠叠加变成层层的包装，PricingService 根据规则编码串构建出这样一条链：折扣策略被补贴装饰器包一层，再被积分装饰器包一层，最后统一被封顶装饰器包一层。新增一种优惠，只需要多写一个装饰器类，计价服务的循环一行不用改。

退款冲正走的是另一条路径：不修改原流水，而是新增一张类型为 REFUND 的单据，原单的状态改为“已冲正”。这样任何一条历史流水都不会被覆盖，审计时能看到完整的来龙去脉。退款时若原单已被冲正，直接返回错误码 1012，防止重复退款。

4.4 开发中遇到的几个问题

下面的四个问题都是实际调试中出现的，记录下来是因为它们分别暴露了一类容易被忽视的错误。

策略实现未被注册。系统启动正常，第一次消费却报空指针，提示 PricingStrategy.calculate 的委托对象为 null。原因是四个计价实现类忘记标注 @Component，而计价上下文是通过集合注入 List<PricingStrategy> 获取实现的，Spring 允许集合为空，因此启动阶段不报错，直到运行时取不到策略才暴露。教训是集合注入的依赖缺失属于延迟失败，需要在启动后主动打一条日志确认实现数量。

Map 结果集没有驼峰转换。账户列表在页面上整列显示问号，按钮也不出现。排查发现 Mapper 返回的是 List<Map>，而 MyBatis 的 mapUnderscoreToCamelCase 只对实体类生效，对 Map 不生效，键名仍是 student_name 这样的下划线形式，前端按 studentName 取值自然取不到。解决方式是在 SQL 里直接写好别名，这类问题在返回 Map 的查询里必须逐条检查。

时间字段格式不统一。接口返回的时间一开始是 2026-11-05T21:46:30 这样的 ISO 格式，通知列表里甚至带上了纳秒，直接展示很难看。配置项 spring.jackson.date-format 只对 java.util.Date 生效，对 LocalDateTime 无效，需要另外注册序列化器。补上一个配置类后，全站时间统一为 yyyy-MM-dd HH:mm:ss。

离线仓库缺少打包插件。本机 Maven 仓库不完整，执行 package 时 surefire 与 jar 插件的大量传递依赖都无法解析。最终放弃打成可执行 jar，改为用 Maven 只做编译，再以 classpath 方式启动，并把所需依赖复制到 lib 目录，配合启动脚本运行。这也让答辩时的现场启动变得简单。

5 测试与结果分析

测试分三层：接口功能测试覆盖正常流程与业务分支，边界与异常测试验证错误码是否符合预期，并发测试验证在竞争条件下余额不会被扣成负数。三部分都用 Python 标准库编写脚本直接调用 HTTP 接口，不依赖任何测试框架，脚本与测试记录一并放在项目的 test 目录下。

5.1 测试环境与方法

表 5-1 测试环境

项目	配置
操作系统	Windows 11
运行环境	JDK 17（IntelliJ 自带运行时）
数据库	MySQL 8.0，库名 campus_card，初始 4 个账户、4 台终端、4 条优惠规则
服务地址	http://localhost:8080
测试工具	Python 3 标准库 urllib 与 threading，脚本化断言
测试数据	每轮测试前重置数据库，保证结果可复现

功能测试共设计 48 个用例，覆盖五个功能模块的正常路径、业务分支与异常输入。每个用例都断言返回体中的 code 与关键字段，全部通过才记为合格。表 5-2 节选了其中有代表性的部分。

5.2 功能测试

表 5-2 功能测试用例（节选）

编号	用例	预期结果	实际	结论
F-01	正常开户	返回成功，生成卡号	卡号 622100000005	通过

F-02	学号重复开户	提示学号已存在	code 1000	通过
F-03	空学号开户	提示学号不能为空	code 1000	通过
F-04	查询不存在的卡	提示账户不存在	code 1001	通过
F-05	普通消费 12.5 元	实付 12.50，优惠 0	实付 12.50	通过
F-06	折扣规则消费 100 元	实付 85.00，优惠 15.00	85.00 / 15.00	通过
F-07	补贴规则消费 20 元	实付 18.00，优惠 2.00	18.00 / 2.00	通过
F-08	积分抵扣消费 30 元	实付 23.85，扣 615 分	23.85 / -615	通过
F-09	折扣与补贴叠加 100 元	实付 83.00，优惠 17.00	83.00 / 17.00	通过
F-10	水控 20 升	按 0.5 元/升计 10.00 元	10.00	通过
F-11	门禁空报文	按次计 1.00 元	1.00	通过
F-12	充值 200 元	余额增加 200	余额正确	通过
F-13	退款冲正	新增退款单，原单转已冲正	金额回补	通过
F-14	重复退款	提示该交易已冲正过	code 1012	通过
F-15	重复请求号	提示该请求已处理	code 1008	通过
F-16	挂失后消费	提示该卡已挂失	code 1002	通过
F-17	有余额时销户	提示先退款再销户	code 1000	通过
F-18	销户后恢复	提示已注销不可恢复	code 1000	通过
F-19	流水分页查询	返回分页数据与总数	total 正确	通过
F-20	按类型筛选流水	只返回该类型	结果正确	通过
F-21	总览统计	各指标与库内数据一致	一致	通过
F-22	七日趋势	返回 7 个数据点	7 个点	通过
F-23	日终结账	生成结算记录	结算表新增记录	通过
F-24	交易通知	四个观察者均被触发	通知与统计同步更新	通过

48 个用例全部通过。其中 F-08 的积分计算值得说明：规则为 100 积分抵扣 1 元，消费 30 元时可用积分抵掉 6.15 元，消耗积分 615 分，实付 23.85 元。这个结果同时验证了计价策略与积分观察者两条链路——前者算扣多少分，后者在交易完成后真正把分从账户里减掉。

5.3 边界与异常测试

边界与异常用例单独整理成表，每一条都对应一个明确的业务规则，返回值必须是设计时约定的错误码，而不是笼统的“系统异常”。

表 5-3 边界与异常测试

场景	输入	预期	错误码
消费金额为 0	amount = 0	消费金额必须大于 0	1000
消费金额为负	amount = -5	消费金额必须大于 0	1000
POS 报文缺金额	payload 为空	POS 报文缺少 amount 字段	1000
支付密码错误	payPwd = 000000	交易密码错误	1005
密码位数不足	新密码 4 位	密码必须为 6 位	1005
余额不足	余额 20，消费 30	余额不足，当前余额 20.00 元	1004
超过单笔限额	消费 300 元	单笔消费不得超过 200.00 元	1006
超过日累计限额	当日累计 500 元	超过日累计限额	1007
终端已停用	终端号对应 status=0	终端不可用	1010
终端不存在	POS-999	终端不可用	1010
冲正不存在的交易	随机交易号	原交易不存在	1011
充值负数	amount = -100	充值金额不合法	1009

这组用例把“边界”落实到了具体数值上：单笔限额取 200 元，测试用 299 元与 201 元验证拒绝，用 199 元与 200 元验证通过；日累计限额取 500 元，先用若

干笔消费把当日累计推到接近上限，再验证下一笔被拒。这些数字都可以在系统参数里调整，测试时从接口读取当前值再构造数据，避免把常量写死在脚本里。

5.4 并发与性能测试

并发测试用 threading.Barrier 让多个线程在同一时刻发起请求，营造真实的竞争条件。共设计两组场景。第一组验证代理的限流能力，同时对同一张卡发起 25 笔 1 元消费；第二组验证余额的原子性，在卡内只有 100 元时并发发起 5 笔 30 元消费，理论上最多只能成功 3 笔。

表 5-4 并发测试结果

场景	并发数	成功	拒绝	拒绝原因	耗时	余额变化
限流校验	25 笔 × 1 元	20	5	操作太频繁，请稍后再试	153 ms	500 → 480
余额竞争	5 笔 × 30 元	3	2	余额不足（并发扣减失败，请重试）	92 ms	100 → 10

第一组的结果符合预期：限流器按“每卡每秒 20 笔”放行，第 21 笔开始被拒，且被拒的请求没有产生任何流水。第二组更能说明问题：100 元余额在 5 个线程同时扣 30 元的条件下，最终只成功 3 笔，余额停在 10 元，既没有出现 4 笔成功导致透支，也没有出现 5 笔全败的保守结果。被拒的两笔返回的是“并发扣减失败”，而不是“余额不足”，说明它们是在数据库层被条件更新挡下的，而不是在责任链的前置校验里被误判。

性能方面，操作日志记录了每笔交易的处理耗时。演示数据中消费类交易平均 33.60 毫秒、最大 60 毫秒，充值 14 毫秒，退款 22 毫秒。这个量级足够支撑校园场景下的实际并发，耗时主要来自三次数据库往返（读卡账户、原子扣减、写入流水），与设计预期一致。

5.5 结果分析

三个层面的测试共同说明了一件事：系统的正确性不是靠前端校验或单点判断保证的，而是靠在数据层把“判断”与“执行”合并成一个原子操作。责任链里的余额校验只负责快速失败与友好提示，真正守住底线的是那条带条件的 UPDATE 语句。这也是并发测试能够稳定复现“恰好 3 笔成功”的原因。

测试也暴露了设计上可以改进的地方。限流阈值最初设为每秒 5 笔，结果正常的演示操作都会被拒，说明阈值必须结合真实使用频率来定，不能凭直觉设一个“安全”的数。另外，当前的对账累计放在内存里，适合单机部署，如果将来扩展成多实例，就需要把观察者的计数下沉到 Redis 或直接在结算表上做增量，这一点在第 7 章还会提到。

6 项目开源

本项目按开源项目的组织方式整理，源码结构可以直接发布到 GitHub（账号 wenciqing）。整理时遵循三条原则：源码目录保持可直接编译的状态，不包含编译产物与本机依赖；数据库脚本单独放在 sql 目录，导入即可运行；README 写清环境要求、启动流程与初始账号，评审者不需要额外的口头说明就能把系统跑起来。

仓库结构如下：

```
campus-card/
├── backend/        后端源码、静态页面、依赖清单
├── sql/            建库脚本与初始数据
├── test/           接口测试、并发测试与测试记录
├── docs/           报告、插图与生成脚本
├── build.sh        离线编译脚本
├── run.bat         启动脚本
└── README.md       运行说明
```

README 是开源项目的门面，本项目在其中写清了四件事：技术栈与版本要求、从建库到启动的三步操作、四个初始账户与四台终端的数据说明、以及九个设计模式各自的落点类。评审者按图索骥就能找到每个模式在代码中的位置，这一点比单纯堆叠功能更容易体现设计意图。

发布前需要处理两件事。一是配置文件脱敏，application.yml 中的数据库密码要换成占位符，改为通过环境变量或本地配置注入，避免把个人凭据推到公开仓库；二是补充 .gitignore，把 target 目录、依赖目录、IDE 配置与日志排除在外，只提交源码与脚本。许可证方面选择 MIT，这是课程项目常用的宽松许可，允许他人自由参考与修改。

7 总结与展望

这次考查选择的题目是校园一卡通消费与结算系统。做完之后回看，这个题目最大的价值不在于功能有多少，而在于它把“变化”摆在了很显眼的位置：优惠规则一定会增加，终端类型一定会增加，校验规则一定会调整，通知渠道一定会扩展。如果一开始就按最直接的方式写 if-else，这些变化每来一次就要动一次主流程，越往后越不敢改。

九种模式里，真正起结构作用的是三个。策略与装饰器负责把“算多少钱”从主流程里拿出去，让优惠的叠加变成对象的层层包装；责任链负责把“能不能扣”拆成六个互不干扰的判断，任何一环出错都不影响其他环；适配器负责把“不同终端说什么话”挡在业务之外，让水控的升数、门禁的空报文、POS 的金额都收敛成同一个请求对象。其余六个模式各有分工：工厂管交易对象的创建，建造者管单据的组装，观察者管交易之后的通知，代理管横切逻辑，单例管全局参数。

收获最大的地方其实不在模式本身，而在两处细节。一处是扣款语句：把判断与扣减写进同一条 UPDATE，靠数据库的行锁来保证原子性，这比在 Java 里加锁要可靠得多，也让并发测试能够稳定通过。另一处是退款的思路：不修改原流水，而是新增反向流水，原单只改状态。这带来的是可审计性——任何一笔账都能顺着原交易号找回完整链路。这两点都不是设计模式教给我的，但它们和设计模式一样，本质上都是在为“以后可能要改”留出余地。

系统还存在不少可以继续完善的地方。对账累计目前放在内存，多实例部署时会各算各的，需要下沉到共享存储；结算目前是手动触发，可以加定时任务在每日零点自动执行；权限体系只做了最基础的角色区分，缺少细粒度的操作授权；前端的图表是手绘的柱状与进度条，数据量上来之后应该换成专门的图表库。这些都是明确的下一步方向。

关于 AI 工具的使用，这里如实说明：本项目在开发过程中使用了 AI 编程助手辅助完成部分代码的编写、调试错误的排查以及报告文字的润色。具体而言，AI 参

与了三类工作——根据需求生成初始的类结构与接口定义、在出现编译或运行错误时协助定位原因、对报告中的表述进行整理。设计模式的选型、数据库表结构的确定、交易主流程的编排顺序、以及并发扣减采用条件更新的方案，都是在明确需求边界后自行决定并逐一验证的。所有 AI 生成的代码都经过了实际运行与测试用例的检验，报告中列出的测试数据均来自真实执行结果，未作虚构。

参考文献

[1] Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides. 设计模式：可复用面向对象软件的基础[M]. 李英军, 马晓星, 蔡敏, 等译. 北京: 机械工业出版社, 2019.

[2] 弗里曼. Head First 设计模式[M]. O'Reilly Taiwan 公司译. 北京: 中国电力出版社, 2007.

[3] Craig Walls. Spring 实战（第 6 版）[M]. 张卫滨译. 北京: 人民邮电出版社, 2022.

[4] Martin Fowler. 企业应用架构模式[M]. 王怀民, 周斌译. 北京: 机械工业出版社, 2010.

[5] MySQL 8.0 Reference Manual: InnoDB Locking and Transaction Model[EB/OL]. https://dev.mysql.com/doc/refman/8.0/en/innodb-locking.html, 2024.

[6] Vue.js 官方文档[EB/OL]. https://cn.vuejs.org/guide/, 2024.

[7] MyBatis 3 中文文档[EB/OL]. https://mybatis.org/mybatis-3/zh/index.html, 2024.